

Adversarial Search: Game Trees and Minimax

Choosing actions when another rational agent controls the response

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Define game state, player function, terminal test, utility, and game tree.
- 02 — Compute minimax values by backward induction.
- 03 — Trace mutually recursive MAX and MIN procedures.
- 04 — Sketch minimax correctness under optimal play.
- 05 — Derive $O(b^m)$ time and $O(bm)$ depth-first space.
- 06 — Design and test depth-limited evaluation functions.

An opponent changes the backup, not the state-space idea

**Minimax values a state by assuming
MAX optimizes utility and MIN
optimizes the opposing objective.**

- MAX chooses the largest child value.
- MIN chooses the smallest child value.
- Terminal utility anchors the recursion.

Contingency

A plan must specify a response to every opponent action, not one fixed path.

Minimax is a recursive value definition

$V(s) = \text{UTILITY}(s)$ if terminal; $\max_a V(\text{RESULT}(s,a))$ for MAX; $\min_a V(\text{RESULT}(s,a))$ for MIN

- $\text{PLAYER}(s)$ identifies who moves.
- $\text{ACTIONS}(s)$ lists legal moves.
- $\text{RESULT}(s,a)$ gives the successor.
- $\text{UTILITY}(s)$ is defined at terminal states.

ENGINEERING IMPLICATION

The root action is selected by child value, not by a single predicted opponent move.

Single-agent paths become adversarial strategies

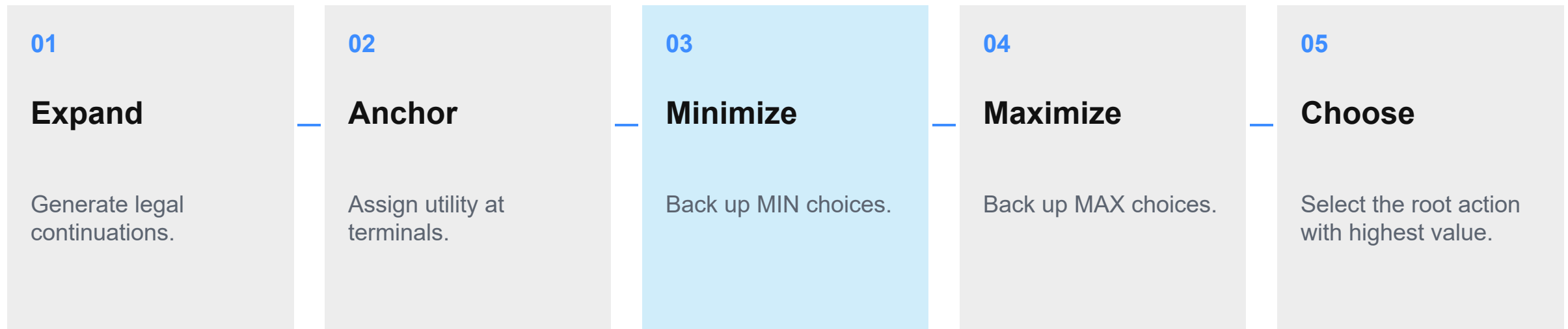
Pathfinding search

- Environment does not choose against the agent.
- Solution is one action sequence.
- Goal and path cost evaluate a path.
- Frontier ordering guides expansion.

Game-tree search

- Players alternate control.
- Solution is a contingent strategy.
- Terminal utility evaluates outcomes.
- Backups propagate values.

Backward induction computes rational play



Every backup represents control by the player named at that state.

Minimax alternates MAX and MIN recursion

```
function MAX_VALUE(s):
    if terminal(s): return utility(s)
    v = -infinity
    for a in actions(s):
        v = max(v, MIN_VALUE(result(s,a)))
    return v

function MIN_VALUE(s):
    if terminal(s): return utility(s)
    v = +infinity
    for a in actions(s):
        v = min(v, MAX_VALUE(result(s,a)))
    return v
```

Required contracts

- Terminal test must stop.
- Utility uses MAX's perspective.
- Turns alternate correctly.
- Ties are deterministic.

Worked minimax backup

SCENARIO

MAX has Left and Right. Their MIN children lead to leaves [3,5] and [2,9].

- Left MIN chooses $\min(3,5)=3$.
- Right MIN chooses $\min(2,9)=2$.
- MAX compares 3 and 2.
- MAX chooses Left.

The root minimax value is 3 even though the tree contains a leaf worth 9.

Game assumptions determine what minimax means

Zero-sum

Opposed utility

- One scalar utility.
- MIN lowers MAX value.
- Standard minimax model.
- Tie policy still matters.

Perfect information

State is visible

- No hidden cards or state.
- Legal actions are known.
- Deterministic successor.
- Chance is absent.

Optimal play

Rational response

- Opponent selects best reply.
- Value is conservative.
- Weak opponents may do better.
- Guarantee is model-relative.

Checkpoint: back up a small tree

CHECKPOINT

A MIN node has three MAX children valued 4, -1, and 6. What value is returned, and which branch can MIN select?

01

Identify the player controlling the node.

02

Apply the correct extremum.

03

Interpret the result from MAX's perspective.

Depth limits replace terminal utility with evaluation

Terminal utility

- Exact for completed games.
- Aligned with win, loss, or score.
- Too deep in large games.
- No approximation at leaves.

Evaluation function

- Estimates nonterminal strength.
- Combines informative features.
- Must use one consistent perspective.
- Creates approximation error.

Game trees grow exponentially

$$\text{nodes through depth } m = 1 + b + \dots + b^m = O(b^m)$$

- b is average legal-move count.
- m is search depth in plies.
- Each ply adds another branching layer.
- Depth-first recursion stores $O(bm)$ pending siblings and path state.

ENGINEERING IMPLICATION

A modest depth increase can multiply work by b , making cutoffs operationally necessary.

Evaluation features encode strategic judgment

SCENARIO

A Pacman-style state is cut off before a terminal win or loss.

- Reward current score and nearby food.
- Penalize collision risk and nearby active ghosts.
- Treat scared ghosts with a different sign.
- Normalize feature scales before weighting.

The evaluator ranks frontier states but does not change the legal game model.

Evaluation defects can dominate deeper search

Perspective error

Sign mismatch

- MIN and MAX disagree.
- Wins can look bad.
- Backups remain consistent but wrong.
- Use one utility convention.

Horizon effect

Consequence beyond cutoff

- Threat is delayed.
- Shallow value looks safe.
- Quiescence can help.
- Test tactical positions.

Scale error

Feature imbalance

- One term overwhelms others.
- Weights become uninterpretable.
- Scores become unstable.
- Inspect component values.

Checkpoint: choose a cutoff result

CHECKPOINT

At a depth-limited MIN node, evaluation values are 7, 3, and 5 from MAX's perspective. What value backs up?

01

Keep the perspective fixed.

02

Apply MIN at the cutoff's parent.

03

State whether the value is exact.

A minimax implementation needs structural tests

Test turn alternation, terminal handling, value perspective, and tie order independently.

Oracle

Tiny hand-valued trees provide exact expected outputs.

- Compare recursive output with a hand-computed tree.
- Assert the input state is not mutated.
- Instrument depth, generated nodes, and chosen root action.

Key takeaways

- 01 ——— Minimax backs values through alternating MAX and MIN control.
- 02 ——— Its result is a contingent strategy under optimal play.
- 03 ——— Time is $O(b^m)$, so practical search uses depth limits.
- 04 ——— Evaluation quality and perspective are part of correctness.

Next: Alpha-beta pruning preserves the minimax value while avoiding irrelevant branches; expectimax adds chance.

Check your understanding

Q1

Back up a two-ply minimax tree.

Q2

Explain why a strategy is not one path.

Q3

Derive $O(b^m)$.

Q4

Design an evaluation function for a small game.

Q5

Name tests for turn and utility defects.